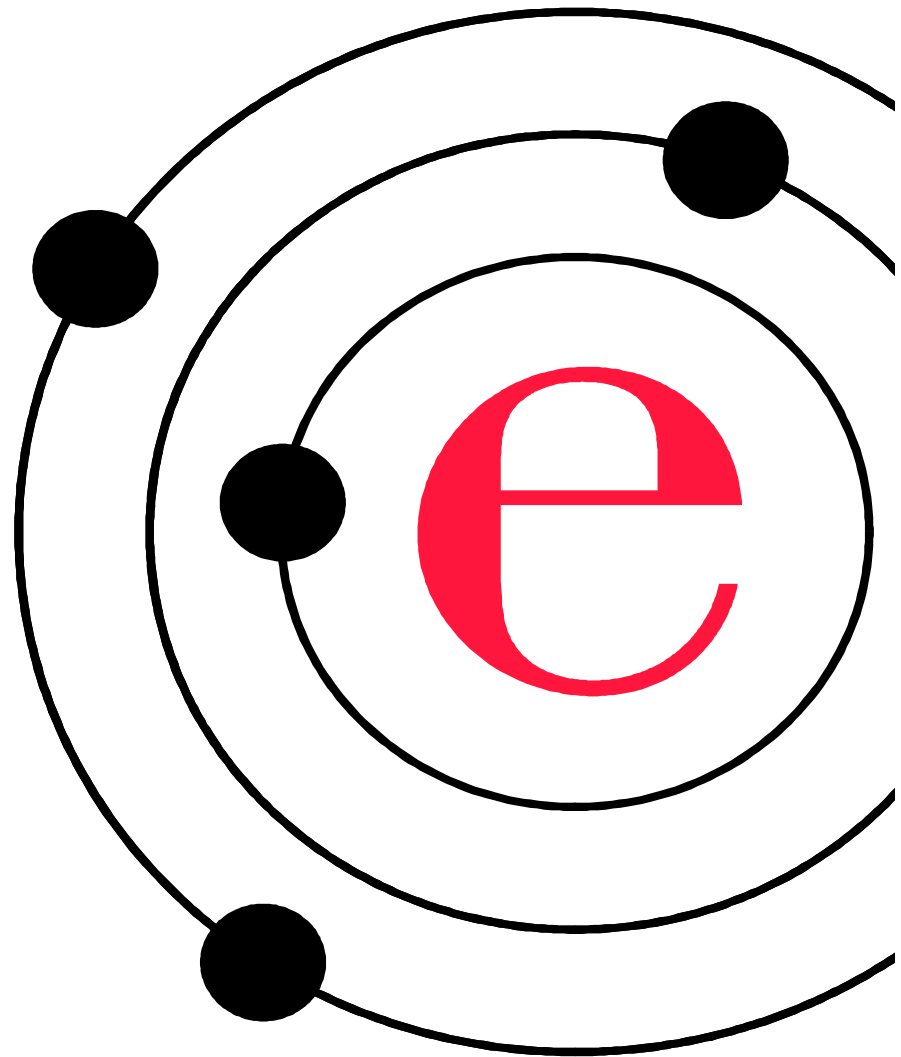


*An overview of
Unicode &
ISO 10646*

Pat Glenski



Abstract

- What is Unicode?
 - Design goals
 - What is ISO-10646
 - Architecture
 - Big or Little ?
 - UCS-2, UCS-4
 - UTF-7, UTF-8
 - UTF-16
- Why Unicode is not a "magic" solution.



ORGANISATION
INTERNATIONALE DE
NORMALISATION



INTERNATIONAL
ORGANIZATION FOR
STANDARDIZATION

Unicode

- ▶ Initial work by Apple and Xerox 1989 as a US corporation. (Not part of the world wide standard bodies)
 - Now supported by all the major industry players
- ▶ Created to overcome problems with
 - encodings that supported different "states"
 - variable length characters
 - encoding scheme varieties
- ▶ 4 original main design goals
 - Universal Large repertoire to encompass all characters that were likely to be used in general text exchange
 - Efficient No need to look for special codes, escape sequences etc.
 - Uniform Fixed code element length allows efficient sorting, searching etc.
 - Unambiguous Distinctive code elements, no overlap on any given 16-bit value



Unicode Architecture

- ▶ This evolved to fundamental principles to guide
 - Atomic size is 16 bit code elements
 - Full 16 bits space available = 65,535 places
 - Encode character, not glyphs
 - character = smallest unit of written language "A"
 - glyph = the shapes these units can take "A" "A" "A" "A"
 - Characters have well defined semantics
 - property tables, sort, parsing, etc.
 - Encodes plain text (not fonts)
 - Does not encode "idiosyncratic, personal, novel, rarely exchanged, or private-use characters, nor logos or graphics"
 - Unifies duplicate characters within scripts across languages
 - Allow for dynamic composition of accent forms
 - Allow static precomposed forms

Unicode & ISO-10646

- ▶ At the same time that Unicode was doing its work in US, ISO group was working on different version of same idea. (ISO is the international standard organization)
- ▶ ISO called its standard ISO-10646
- ▶ After Unicode published its first recommendation in 1990 and ISO published its 1st draft the two got together in 1993 and agreed on only 1 definition for code values.
- ▶ The Unicode Standard, Version 2.0 is code value for code value equivalent with the Universal Character Set, two-octet form (UCS-2) of ISO/IEC 10646.
 - As the UCS-2 subset of ISO 10646, the Unicode Standard's 65,536 code values are the first 65,536 code values of ISO 10646..

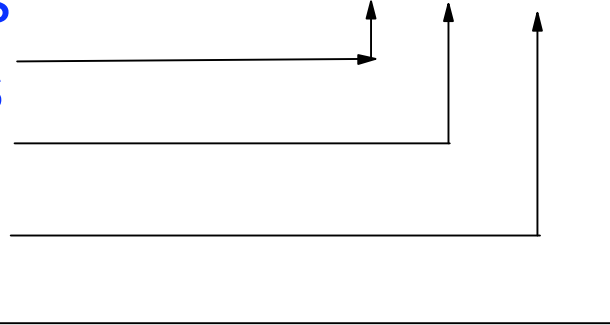
ISO-10646 Standard

- ▶ Basic structure is 4 bytes per character
- ▶ Sets the hex value for each character

- ▶ groups, planes, rows, cells

- 256 groups, each has
 - 256 planes, each has
 - 256 rows, each has
 - 256 cells

So 00 00 00 41 is "A"



- ▶ Group 0, plane 0, is the BMP (2 bytes)
this is 65,535 places.
0000xxyy

ISO 10646 Defines

► Form desired (UCS-2 or UCS-4)

- allows for a 2 byte version ISO-10646 UCS-2
"A" is 0041x
- allows for a 4 byte version ISO-10646 UCS-4
"A" is 00000041x

► Implementation level

1. precomposed characters only Á
2. restricted combinations, only 300 or so
3. unrestricted combinations, any allowed

Example of composing

Composed	Base	Accent
Á	A	´
Ç	C	¸
Ô	O	^
Ü	U	..

ISO 10646 Defines

- ▶ So ISO allows the following "official versions"
 - ISO-10646 UCS-4
 - ISO-10646 UCS-2
 - ISO-10646 UCS-4 level 1
 - ISO-10646 UCS-2 level 1
 - ISO-10646 UCS-4 level 2
 - ISO-10646 UCS-2 level 2
 - ISO-10646 UCS-4 level 3
 - ISO-10646 UCS-2 level 3

- ▶ so to say product XYZ supports "unicode" or "iso-10646" can be vague

Unicode Defines

- ▶ Behavior and use of characters
- ▶ A BiDi algorithm
- ▶ Order and use of non-spacing marks, equivalence rules
- ▶ Mappings for compatibility characters
- ▶ Default shaping for cursive scripts
- ▶ Conversion maps
- ▶ Directional attributes
- ▶ Limited to only the BMP of ISO-10646 (sort of)

ISO defines "the road"
Unicode is "The rules of the road"

Unicode Architecture

- The Unicode Standard defines codes for characters used in every major language written today.
- Scripts include Latin, Greek, Cyrillic, others
- a unified set of Chinese/Japanese/Korean (CJK) ideographs.

Unicode Architecture

- CJK examples, U+4E97 and U+5176

	Chinese				Japanese		Korean
Unicode	GB 2312	CNS 11643	CCCII	Big Five	JIS X 0208	JIS X 0212	KS C 5601
							

	Chinese				Japanese		Korean
Unicode	GB 2312	CNS 11643	CCCII	Big Five	JIS X 0208	JIS X 0212	KS C 5601
							

Unicode Architecture

► Layout is by Character Blocks

- General Scripts Area: {U+0000..U+1FFF}
 - Basic Latin (US-ASCII): {U+0000..U+007F}
 - Latin-1 (ISO-8859-1): {U+0080..U+00FF}
 - Spacing Modifier Letters: {U+02B0..U+02FF}
 - Combining Diacritical Marks: {U+0300..U+036F}
 - Greek: {U+0370..U+03FF}
 - Cyrillic: {U+0400..U+04FF}
 - Arabic: {U+0600..U+06FF}

► The Unicode numbering is supposed to be logical with zones and blocks and ranges of related characters grouped together.

However, as blocks turned out to be too small, scripts had to be extended at remote encoding blocks so that Latin, Greek, Hebrew, Arabic, Hangul, and Han are now split across several blocks which makes certain characters harder to find and certain aspects of the Unicode ordering more historical than logical.

- Private Use Area
- Compatibility Area and Specials

Big or Little Endian

- ▶ In C code, needed to hold Unicode (UCS-2) this looked like integer, so used a 2 byte integer. (in Java it is always big endian)
- on non-Intel machines this is stored as big endian format (This is the **default** specified by ISO)
- on Intel machines, this is stored in little endian format
 - reserved 2 special codes to indicate
 - FEFF called byte order mark or "BOM"
(and FFEE is reserved)

The use of BOM is NOT required

so in memory (on the disk) "ABC" could stored like

- Big 00,41,00,42,00,43
- Little 41,00,42,00,43,00

UTF-7 UTF-8

- ▶ If you look at a UCS-2 string from a C point of view it is full of nulls, so lots of c string functions "don't work"

"abc" = 006100620063 in hex
strlen of this is 0 bytes long not 6.

- ▶ Lots of file systems and transport systems were based on C functions and so can not handle '00'x imbedded in strings

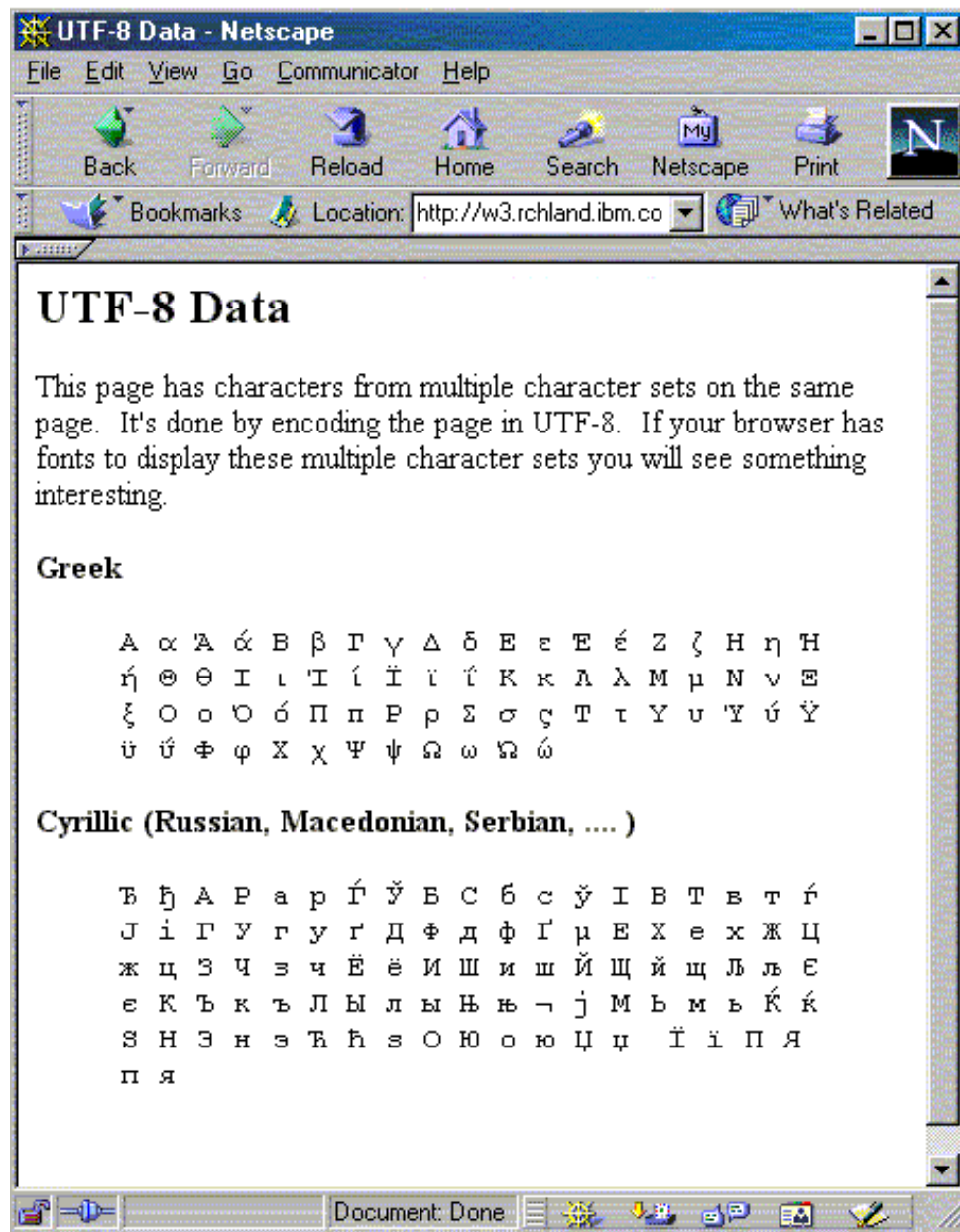
UTF-7 UTF-8

- ▶ To solve, created UTF-x, started as FSS (file system safe) now called UCS Transformation Format
 - ▶ Converts (via an algorithm) UCS data to
 - no longer have nulls in it and
 - to have either 7 or 8 data bits used.
 - UTF-7 keeps a-z and A-Z and 0-9 as plain ASCII codes
 - UTF-8 keeps all ASCII codes from 00 to 7F as themselves
 - converts the others to 2,3,4 bytes long
- Example:
- 0000 (null) becomes 00
 - 0041 (A) becomes 41
 - 009A (copyright) becomes C2A9
 - 2260 (not equal) becomes E289A0

UTF-8

- ▶ is used to store Unicode on several UNIX platforms
- ▶ is used as the default encoding for most new internet standards
- ▶ UTF-8 has some advantages
 - ✓ transparency for ASCII characters 00-7F hex.
 - ✓ BOM avoidance
 - ✓ reasonably compact encoding ASCII characters are not inflated, most other alphabetic characters occupy only two bytes each
 - ✓ processor-friendliness read and written quickly with simple bitmask and bitshift
 - ✓ self-synchronization you can always distinguish a lead byte 11vvvvv from a fill byte 10vvvvv
 - ✓ canonical sort-order Comparing two UTF-8 multibyte character strings with the old `strcmp(mbs1,mbs2)` function gives the same result as the `wcscmp(wcs1,wcs2)` function on the corresponding UCS-4
- ▶ Some disadvantages
 - ✗ Not fixed width Char(10) ??
 - ✗ processor cycles used
 - ✗ can need up to 4 bytes for 1 character

UTF-8 Support in Browsers



UTF-8 is the "new" default internet encoding for XML, Webpages etc.

Netscape:
View, Character set, UTF-8

IE:
View, Encoding, More,
Unicode (UTF-8)

Miscellaneous

- ▶ Characters have full names like

A LATIN CAPITAL LETTER A

or

৳ BENGALI CURRENCY NUMERATOR ONE LESS THAN
THE DENOMINATOR

- ▶ Also use U+xxxx to refer like
U+0041 or U+0958
- ▶ Private use area, U+E000 to U+F8FF
- ▶ Process code & File code

Surrogates & UTF-16

- ▶ After Unicode was set in 1992 Unicode found out that picking 2 bytes was too small of a space
- ▶ Unicode created the idea of Surrogates
This is a reserved set of pairs of 2 bytes each
 - a high order surrogate U+D800 -> U+DBFE
 - a low order surrogate U+DC00 -> U+DFFE

So, this would mean 1 character would need 4 bytes
(2+2)

- ▶ This allows to encode over 1,000,000 more spaces
 - No codes defined yet, just the way to do it.
- ▶ **UTF-16** is "normal" 2 byte Unicode + the use of surrogates
- ▶ **UTF-32** is "normal" UCS-4 but limited to the range of UTF-16.

Why Unicode is not the (complete) perfect answer

- ▶ unicode makes programming MUCH easier!
 - no more need to track 50+ codes, just convert to unicode on input, do your logic, then convert to xxx for output.
- ▶ Unicode/ISO 10646 solves the question on how to "keep" the characters safe.
- ▶ Does not address how to enable code for global use. Still need to handle sorting, date format etc.

Unicode Layout 1

A-zone (alphabetic): {U+0000..U+33FF}

General Scripts Area: {U+0000..U+1FFF}

Basic Latin (US-ASCII): {U+0000..U+007F}

Latin-1 (ISO-8859-1): {U+0080..U+00FF}

Latin Extended: {U+0100..U+024F}

IPA Extensions: {U+0250..U+02AF}

Spacing Modifier Letters: {U+02B0..U+02FF}

Combining Diacritical Marks: {U+0300..U+036F}

Greek: {U+0370..U+03FF}

Cyrillic: {U+0400..U+04FF}

Armenian: {U+0530..U+058F}

Hebrew: {U+0590..U+05FF}

Arabic: {U+0600..U+06FF}

Syriac: {U+0700..U+074D}

Thaana: {U+0780..U+07B1}

ISCII Indic Scripts: {U+0900..U+0DFF}

Devanagari: {U+0900..U+097F}

Bengali: {U+0980..U+09FF}

Gurmukhi: {U+0A00..U+0A7F}

Gujarati: {U+0A80..U+0AFF}

Oriya: {U+0B00..U+0B7F}

Tamil: {U+0B80..U+0BFF}

Telugu: {U+0C00..U+0C7F}

Kannada: {U+0C80..U+0CFF}

Malayalam: {U+0D00..U+0D7F}

Sinhalese: {U+0D80..U+0DFF}

Thai: {U+0E00..U+0E7F}

Lao: {U+0E80..U+0EFF}

Tibetan: {U+0F00..U+0FBF}

Mongolian: {U+1000..U+109F}

Georgian: {U+10A0..U+10FF}

Hangul Jamo: {U+1100..U+11FF}

Ethiopic: {U+1200..U+137F}

Cherokee: {U+13A0..U+13FF}

Canadian Syllabics: {U+1400..U+167F}

Ogham: {U+1680..U+169F}

Runic: {U+16A0..U+16FF}

Burmese: {U+1700..U+1759}

Khmer: {U+1780..U+17E9}

Latin Extended Additional: {U+1E00..U+1EFF}

Greek Extended: {U+1F00..U+1FFF}

Unicode Layout 2

Symbols Area: {U+2000..U+2EFF}

General Punctuation: {U+2000..U+206F}

Superscripts and Subscripts: {U+2070..U+209F}

Currency Symbols: {U+20A0..U+20CF}

Combining Marks for Symbols: {U+20D0..U+20FF}

Letterlike Symbols: {U+2100..U+214F}

Number Forms: {U+2150..U+218F}

Arrows: {U+2190..U+21FF}

Mathematical Operators: {U+2200..U+22FF}

Miscellaneous Technical: {U+2300..U+23FF}

Control Pictures: {U+2400..U+243F}

Optical Character Recognition: {U+2440..U+245F}

Enclosed Alphanumerics: {U+2460..U+24FF}

Box Drawing: {U+2500..U+257F}

Block Elements: {U+2580..U+259F}

Geometric Shapes: {U+25A0..U+25FF}

Miscellaneous Symbols: {U+2600..U+26FF}

Dingbats: {U+2700..U+27BF}

Braille Pattern Symbols: {U+2800..U+28FF}

CJK Phonetics and Symbols Area: {U+2F00..U+33FF}

KangXi radicals: {U+2F00..U+2FD5}

CJK Symbols and Punctuation: {U+3000..U+303F}

Hiragana: {U+3040..U+309F}

Katakana: {U+30A0..U+30FF}

Bopomofo: {U+3100..U+312F}

Hangul Compatibility Jamo: {U+3130..U+318F}

Kanbun: {U+3190..U+319F}

Enclosed CJK Letters and Months: {U+3200..U+32FF}

CJK Compatibility: {U+3300..U+33FF}

I-zone (ideographic): {U+3400..U+9FFF}

CJK Unified Ideographs, Extension A: {U+3400..U+4DFF}

(that range held Hangul syllables in Unicode 1, was undefined in Unicode 2, and will hold ideographs in Unicode 3).

CJK Unified Ideographs: {U+4E00..U+9FA5}

O-zone (other): {U+A000..U+D7FF}

Yi: {U+A000..U+A4C8}

Hangul syllables: {U+AC00..U+D7A3}

Unicode Layout 3

S-zone (surrogates): {U+D800..U+DFFF}

High Surrogates {U+D800..U+DBFF}

Low Surrogates {U+DC00..U+DFFF}

R-zone (reserved): {U+E000..U+FFFD}

Private Use Area: {U+E000..U+F8FF}

Compatibility Area and Specials: {U+F900..U+FFFF}

CJK Compatibility Ideographs: {U+F900..U+FAFF}

Alphabetic Presentation Forms: {U+FB00..U+FB4F}

Arabic Presentation Forms-A: {U+FB50..U+FDFF}

Combining Half Marks: {U+FE20..U+FE2F}

CJK Compatibility Forms: {U+FE30..U+FE4F}

Small Form Variants: {U+FE50..U+FE6F}

Arabic Presentation Forms-B: {U+FE70..U+FEFF}

Halfwidth and Fullwidth Forms: {U+FF00..U+FFEF}

Specials: {U+FFF0..U+FFFF}

UTF-16 extra planes (to be filled by Unicode 4 and ISO-10646-2)

Non-Han Supplementary Plane 1: {U-00010000..U-0001FFFF}

Etruscan: {U-00010200..U-00010227}

Gothic: {U-00010230..U-0001024B}

Klingon: {U-000123D0..U-000123F9}

Western Musical Symbols: {U-0001D103..U-0001D1D7}

Han Supplementary Plane 2: {U-00020000..U-0002FFFF}

Reserved Planes 3..13: {U-00030000..U-000DFFFF}

Plane 14: {U-000E0000..U-000EFFFF}

Language Tag Characters: {U-000E0000..U-000E007F}

Private Use Planes: {U-000F0000..U-0010FFFF}

UCS Information

► URL

General info

<http://www.unicode.org> Unicode page

<http://www.iso.ch> ISO page

*Introduction to
Unicode &
ISO 10646*

Pat Glenski

